

Стиск зображень з використанням дискретних косинусних перетворень

Методичні вказівки

до лабораторної роботи № 3 з дисципліни “Комп’ютерні засоби обробки
сигналів” для студентів спеціальності "Комп'ютерні системи та мережі"

Ужгород– 2024

Мета роботи

Опрацювати та випробувати в середовищі MATLAB 6.0 програму, яка реалізує етапи стиску зображень з використанням дискретного косинусного перетворення.

Теоретичне підґрунтя

Безвтратні методи стиску (кодування Хафмена, LZW, довжин серій) не забезпечують потрібного у багатьох випадках ступеня стиску зображень. Необхідно застосовувати методи стиску з втратою інформації. Один з таких підходів використовується у форматах стиску зображень JPEG.

Стиск даних у форматі JPEG (Joint Photographic Experts Group), який дозволяє стискати окремі (незмінні, still picture) зображення, можна умовно розбити на три етапи:

1-й етап - перетворення та субдискретизація кольорової інформації.

2-й етап – поблочні дискретні косинусні перетворення.

3-й етап – квантування та кодування значень дискретного косинусного перетворення.

Перший етап - це перетворення та субдискретизація кольорової інформації. Він полягає в наступному.

Кожна точка зображення, представлена 3 байтами в системі RGB, переводиться в систему YUV (яскравість, кольорова насиченість, кольоровий тон) згідно виразів:

$$\begin{aligned}Y &= 27/256 * R + 150/256 * G + 29/256 * B \\U &= 131/256 * R - 110/256 * G - 21/256 * B + 128 \\V &= -44/256 * R - 87/256 * G + 131/256 * B + 128\end{aligned}$$

або в матричному вигляді

$$\begin{pmatrix} Y \\ U - 128 \\ V - 128 \end{pmatrix} = \begin{pmatrix} 0,301 & 0,586 & 0,113 \\ 0,512 & -0,430 & -0,082 \\ -0,172 & -0,340 & 0,512 \end{pmatrix} \begin{pmatrix} R \\ G \\ B \end{pmatrix}$$

Перетворення з системи YUV в систему RGB виконується за формулами:

$$\begin{aligned}R &= Y + 1,37 * (U - 128) \\G &= Y - 0,698 * (U - 128) - 0,336 * (V - 128) \\B &= Y + 1,73 * (V - 128)\end{aligned}$$

або в матричному вигляді

$$\begin{pmatrix} R + 175,4 \\ G - 132,4 \\ B + 221,4 \end{pmatrix} = \begin{pmatrix} 1 & 1,370 & 0 \\ 1 & -0,698 & -0,336 \\ 1 & 0 & 1,730 \end{pmatrix} \begin{pmatrix} Y \\ U \\ V \end{pmatrix}$$

Далі значення компоненти Y залишаються без зміни, а число значень компонент U і V зменшується (так звана субдискретизація; компоненти U і V можна загрубити без суттєвої втрати якості зображення). Можливі різні варіанти субдискретизації: просте викидання частини з сусідніх точок чи заміна значень сусідніх точок зображення на їх середні. При цьому можливі кілька варіантів об'єднання точок: дві по горизонталі, дві по вертикалі, квадрат з чотирьох сусідніх точок. Найчастіше використовується наступний

варіант: число точок зменшується вдвоє, причому значення точок обчислюються згідно з виразом

$$y(n) = 1/4 * x(n-2) + 1/2 * x(n-1) + 1/4 * x(n).$$

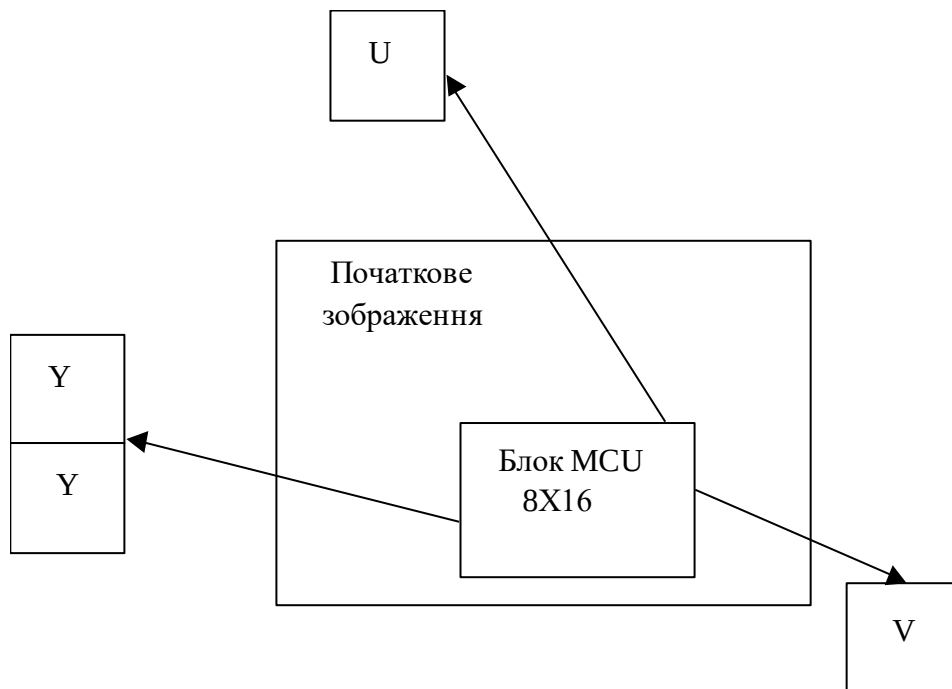
При цьому блок 8X16 значень компоненти U або V перетворюється в блок 8X8 значень.

При відтворенні інформації для покращення якості проміжні точки рекомендується отримувати не простим повторенням, а шляхом інтерполяції між сусідніми точками. Найчастіше використовується наступний спосіб: при відтворенні зображення блок 8X8 точок реконструюється в блок 8X16 точок за формулою

$$x(n) = [y(2n) + y(2n-1)] / 2.$$

Якщо використовується описаний варіант субдискретизації, то досягається стиск зображення в 1,5 раза. Дійсно 1 байт компоненти яскравості залишається без змін, а кожні 2 байти компонент U і V замінюються на 1 байт. Отже, замість 6 байт на кожних 2 точки зображення тепер припадає 4 байта. Якщо використовується варіант об'єднання чотирьох сусідніх точок, то досягається стиск зображення в 2 рази. Адже 1 байт компоненти яскравості залишається без змін, а кожні 4 байти компонент U і V замінюються на 1 байт. Тобто, замість 12 байт на кожних 4 точки зображення тепер припадає 6 байт.

Мінімальний фрагмент інформації (MCU) для обробки – це блок початкового RGB зображення розміру 8X16 елементів. У результаті обробки такого фрагменту на першому етапі отримуємо чотири блоки розміру 8X8: два блоки розміру 8X8 для компоненти яскравості Y та по одному блоку розміру 8X8 для компонент U і V. Це ілюструється наступним рисунком.



Другий етап має в своїй основі дискретне косинусне перетворення (ДКП).

Кожна з компонент Y, U, V зображення на цьому етапі розглядається як окреме монохромне (однокольорове) зображення і її стиск проводиться окремо.

Зображення розбивається на блоки 8X8 елементів. До кожного блоку P застосовується двовимірне ДКП

$$P_{ДКП} = A P A^T,$$

де A - матриця двовимірного ДКП;

$P_{ДКП}$ - матриця значень ДКП фрагменту зображення.

Пряме ДКП задається наступним виразом

$$BD(u, v) = \frac{1}{4} c(u) c(v) \sum_{y=0}^7 \sum_{x=0}^7 bd(x, y) \cos \frac{\pi u(2x+1)}{16} \cos \frac{\pi v(2y+1)}{16}$$

а обернене ДКП задається формулою

$$bd(x, y) = \frac{1}{4} \sum_{y=0}^7 \sum_{x=0}^7 c(u) c(v) BD(u, v) \cos \frac{\pi u(2x+1)}{16} \cos \frac{\pi v(2y+1)}{16}$$

де $c(i) = \frac{1}{\sqrt{2}}$ для $i=0$ та $c(i)=1$ для $i=1, 2, \dots, 7$.

Двовимірне ДКП має ту властивість, що воно зосереджує найбільші значення у верхньому лівому куті матриці перетворення. Типовий розподіл ДКП коефіцієнтів показано в наступній матриці:

500	100	100	100	0	0	0	0
100	100	100	100	0	0	0	0
100	100	100	100	0	0	0	0
100	100	100	100	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	01	0	0	0	0	0
0	0	0	0	0	0	0	0

Ці значення треба взяти більш точно, а решта значень ДКП можна суттєво закругити. Це і є основою стиску зображень у форматі JPEG. Дії, які реалізують описану ідею, відбуваються на наступному третьому етапі.

На третьому етапі виконуються квантування та кодування значень дискретного косинусного перетворення.

Спочатку виконується квантування значень ДКП. Для цього формується матриця Q дільників з елементами $q(i, j) = 1 + (1 + i + j)g$, $i, j = 0, 1, \dots, 7$; g - параметр, який впливає на якість зображення, що отримуємо при відтворенні. Для компонент Y рекомендується брати $g=2$, а для компонент U, V значення g може бути більшим. $q(i, j)$ це не що інше як крок квантування, який залежно від позиції змінюється. При русі від верхнього лівого кута до правого нижнього кута крок квантування збільшується, тобто виконується грубіше квантування.

При $g=2$ матриця дільників має вигляд

3	5	7	9	11	13	15	17
5	7	9	11	13	15	17	19
7	9	11	13	15	17	19	21
9	11	13	15	17	19	21	23
11	13	15	17	19	21	23	25
13	15	17	19	21	23	25	27
15	17	19	21	23	25	27	29
17	19	21	23	25	27	29	31

Значення $x(i,j)$ матриці $P_{ДКП}$ діляться на відповідні значення $q(i,j)$ матриці дільників і заокруглюються до найближчого цілого. Процес квантування описується наступним виразом:

$$Q(x(i,j),i,j)=round(x(i,j)/q(i,j)).$$

Приклад, як виглядає матриця ДКП після квантування значень, наведено нижче:

30	0	0	0	0	0	0	0
-7	-8	1	1	0	0	0	0
-11	6	0	1	0	0	0	0
-5	-3	0	0	0	0	0	0
-7	-3	2	0	0	0	0	0
-4	4	0	0	0	0	0	0
-1	0	1	0	0	0	0	0
-3	1	0	0	0	0	0	0

До коефіцієнтів ДКП, які знаходяться на першому місці (DC коефіцієнти), застосовується дельта імпульсно-кодova модуляція:

$$DC(-1)=0; del(0)=0; del(n)=DC(n)-DC(n-1)$$

До коефіцієнтів, які знаходяться на місцях крім першого (AC коефіцієнти), застосовується описане нижче кодування.

Останнім кроком є застосування до елементів отриманої матриці кодування Хафмена разом з кодуванням довжин серій для послідовностей нулів. Вважається, що поява нульового елемента у даній матриці є найбільш імовірною, а із збільшенням абсолютної величини елемента ймовірність його появи зменшується. Один з варіантів таблиці кодування виглядає наступним чином.

елемент	кодове слово
0	1
+1	0100
-1	0101
+2	0110
-2	0111
+3	00100
-3	00101
+4	00110
-4	00111
+5	0001000
-5	0001001
+6	0001010
-6	0001011
+7	0001100
-7	0001101
+8	0001110
-8	0001111
$ D >8$	00001+8розрядів значення D

Таблиця кодування може фіксуватися перед початком обробки або бути адаптивною, тобто мінятися з використанням статистики появи даних у попередніх блоках.

Кодування матриці квантованих елементів ДКП проводиться по шляху, показаному послідовними номерами (так званий зигзаг або змійка; у напрямку збільшення значень елементів матриці дільників).

0	1	5	6	14	15	27	28
2	4	7	13	16	26	29	42
3	8	12	17	25	30	41	43
9	11	18	24	31	40	44	53
10	19	23	32	39	45	52	54
20	22	33	38	46	51	55	60
21	34	37	47	50	56	59	61
35	36	48	49	57	58	62	63

Результатом кодування є послідовність кодових слів коду Хафмена. Відомо, що ніякі два слова в цьому коді не мають однакового початку, тобто слова в послідовності можуть іти підряд без маркерів розділення.

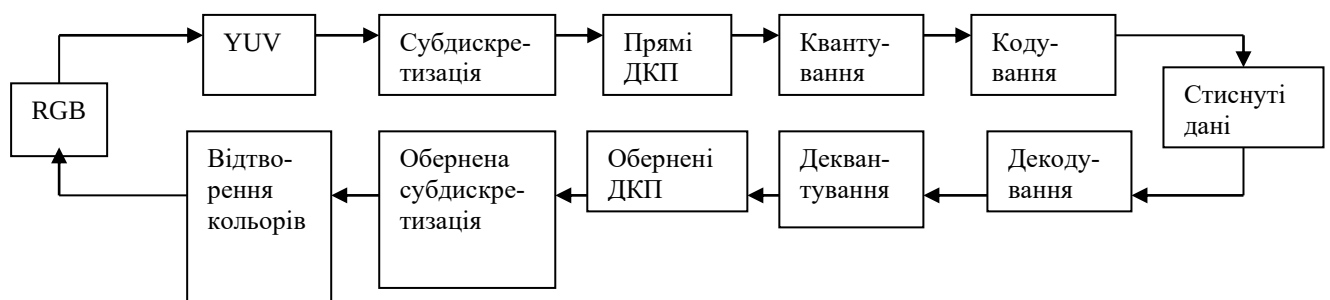
Певні маркери мають бути лише для того, щоб вказати, де починається послідовність кодових слів, отриманих при обробці блоку 8X8 елементів.

Замість застосування коду Хафмена стандарт допускає також використання безвтратних арифметичних кодів.

При відтворенні зображення виконуються наступні дії, зворотні до дій при стиску:

- декодування Хафмена;
- деквантування (множення на відповідні значення елементів матриці дільників);
- обернені двовимірні ДКП блоків 8X8 елементів;
- обернена кольорова субдискретизація (відтворення проміжних точок для компонент U,V шляхом інтерполяції між сусідніми точками);
- перехід від системи YUV до системи RGB.

Етапи стиску та відтворення даних при використанні формату JPEG показані на рисунку.



Степінь стиску в форматі JPEG - від 16:1 до 25:1 без помітної втрати якості.

Зразок програми на мові сценаріїв MATLAB 6.0

Далі наведено зразок програми на мові сценаріїв MATLAB 6.0, яка реалізує описані вище етапи 1-3 стиску зображень.

```

clear all;
imshow source.tif;
im_rgb=imread('source.tif');
info='Started JPEG compression'

% RGB to YUV
info='RGB to YUV...'
a_rgb=double(im_rgb(1:400,1:496,1:3));
a_y=round(77/256*a_rgb(1:400,1:496,1)+150/256*a_rgb(1:400,1:496,2)+29/256*a_rgb(1:400,1:496,3));
a_u=round(131/256*a_rgb(1:400,1:496,1)-110/256*a_rgb(1:400,1:496,2)-21/256*a_rgb(1:400,1:496,3)+128);
a_v=round(-44/256*a_rgb(1:400,1:496,1)-87/256*a_rgb(1:400,1:496,2)+131/256*a_rgb(1:400,1:496,3)+128);
%info='finished'

% downsampling
info='downsampling U,V...'
a_u_ds(1:400,1)=a_u(1:400,1);
a_v_ds(1:400,1)=a_v(1:400,1);

for j=1:400
    for i=2:248
        a_u_ds(j,i)=round(a_u(j,2*i-2)/4+a_u(j,2*i-1)/2+a_u(j,2*i)/4);
        a_v_ds(j,i)=round(a_v(j,2*i-2)/4+a_v(j,2*i-1)/2+a_v(j,2*i)/4);
    end;
end;
clear a_u a_v
%info='finished'

%dct
info='2-D Discrete Cosine Transform ...'
y_dct=blkproc(a_y,[8,8],'dct2');
u_dct=blkproc(a_u_ds,[8,8],'dct2');
v_dct=blkproc(a_v_ds,[8,8],'dct2');
clear a_u_ds a_v_ds
%info='finished'

%quantization
info='quantization ...'
r=3;
for i=1:8
    for j=1:8
        kvant(i,j)=1+(i+j-1)*r;
    end;
end;

for j=1:50
    for i=1:62
        y_kv(8*j-7:8*j,8*i-7:8*i)=round(y_dct(8*j-7:8*j,8*i-7:8*i)./kvant);
    end;
end;
for j=1:50
    for i=1:31
        u_kv(8*j-7:8*j,8*i-7:8*i)=round(u_dct(8*j-7:8*j,8*i-7:8*i)./kvant);
        v_kv(8*j-7:8*j,8*i-7:8*i)=round(v_dct(8*j-7:8*j,8*i-7:8*i)./kvant);
    end;
end;
clear fuctions;

```

```

clear y_dct u_dct v_dct;
%info='finished'

%Differential pulse-code modulation (DPCM) coding and runlength coding
info='Zigzag Ordering Y ...'
for j=1:50
    for i=1:62
        y_mas((j-1)*62+i,1:64)=zigzag(y_kv(8*j-7:8*j,8*i-7:8*i));
    end;
end;
info='Zigzag Ordering U,V ...'
for j=1:50
    for i=1:31
        u_mas((j-1)*31+i,1:64)=zigzag(u_kv(8*j-7:8*j,8*i-7:8*i));
        v_mas((j-1)*31+i,1:64)=zigzag(v_kv(8*j-7:8*j,8*i-7:8*i));
    end;
end;
clear fuctions;
info='Coding Y (DPCM & RunLength) ...'
x_y=0;midle=[];
for i=1:3100
    y_cod=[midle y_mas(i,1)-x_y runcode(y_mas(i,2:64))];
    x_y=y_mas(i,1);
    midle=y_cod;
end;
clear functions;
info='Coding U,V (DPCM & RunLength) '
x_u=0;x_v=0;midle_u=[];midle_v=[];
for i=1:1550
    u_cod=[midle_u u_mas(i,1)-x_u runcode(u_mas(i,2:64))];
    x_u=u_mas(i,1);
    midle_u=u_cod;
    v_cod=[midle_v v_mas(i,1)-x_v runcode(v_mas(i,2:64))];
    x_v=v_mas(i,1);
    midle_v=v_cod;
end;
clear y_kv u_kv v_kv;
clear functions;

info='Huffman coding Y ...'
m_y=0;
for i=1:length(y_cod)
    m_y=m_y+huffman(y_cod(i));
end;
clear functions;
info='Huffman coding U,V ...'
m_u=0;
for i=1:length(u_cod)
    m_u=m_u+huffman(u_cod(i));
end;
clear functions;
m_v=0;
for i=1:length(v_cod)
    m_v=m_v+huffman(v_cod(i));
end;
clear functions;

%info='finished coding'
info='Finished JPEG Compression'

```



```

m=size(im_rgb);
compression_ratio=8*m(1)*m(2)*m(3)/(m_y+m_u+m_v);
%clear m m_y m_u m_v
compression_ratio
imshow source.tif;
%clear all;
function [res]=zigzag(x)
res=[x(1,1) x(1,2) x(2,1) x(3,1) x(2,2) x(1,3) x(1,4) x(2,3) x(3,2) x(4,1)
x(5,1) x(4,2) x(3,3) x(2,4) x(1,5) x(1,6) x(2,5) x(3,4) x(4,3) x(5,2) x(6,1)
x(7,1) x(6,2) x(5,3) x(4,4) x(3,5) x(2,6) x(1,7) x(1,8) x(2,7) x(3,6) x(4,5)
x(5,4) x(6,3) x(7,2) x(8,1) x(8,2) x(7,3) x(6,4) x(5,5) x(4,6) x(3,7) x(2,8)
x(3,8) x(4,7) x(5,6) x(6,5) x(7,4) x(8,3) x(8,4) x(7,5) x(6,6) x(5,7) x(4,8)
x(5,8) x(6,7) x(7,6) x(8,5) x(8,6) x(7,7) x(6,8) x(7,8) x(8,7) x(8,8)];

function [res]=runcode(vect)
n=0;res=[];
for i=1:63
    if vect(i)==0
        n=n+1;
    else
        res=[res n vect(i)];
        n=0;
    end;
end;
res=[res 0 0];

function kod=huffman(elem)
switch elem
case 0, kod=1; % kode word=1
case 1, kod=4; % kode word=0100
case -1, kod=4; % kode word=0101
case 2, kod=4; % kode word=0110
case -2, kod=4; % kode word=0111
case 3, kod=5; % kode word=00100
case -3, kod=5; % kode word=00101
case 4, kod=5; % kode word=00110
case -4, kod=5; % kode word=00111
case 5, kod=7; % kode word=0001000
case -5, kod=7; % kode word=0001001
case 6, kod=7; % kode word=0001010
case -6, kod=7; % kode word=0001011
case 7, kod=7; % kode word=0001100
case -7, kod=7; % kode word=0001101
case 8, kod=7; % kode word=0001110
case -8, kod=7; % kode word=0001111
otherwise kod=13; % kode word=00001+elem(8 bits)

end;

```

Типове зображення, яке обробляємо – файл зображення з розширенням tiff.
(ВЛАСНЕ ФОТО)

Зміст звіту

1. Мета роботи.
2. Основні теоретичні відомості.

3. Блок-схема алгоритму стиску.
4. Результат виконання індивідуального завдання
 - а) скоректована програма на мові сценаріїв MATLAB 6.0, яка реалізує етапи стиску зображень з використанням дискретного косинусного перетворення;
 - б) потік бітів, які є результатом опрацювання заданого викладачем зображення;
 - в) оцінка ступеня стиску зображення.
5. Висновок.

ЛІТЕРАТУРА

1. Бабак В.П., Хандецький А.І., Шрюфер Е. Обробка сигналів: підручник для вузів., К., Либідь, 1996.- 390с.
2. Бондарев В.Н., Трестер Г., Чернега В.С. Цифровая обработка сигналов: методы и средства. - Харьков: Конус, 2001 (підручник для вузів).